ЗАДАНИЕ 1

1.1. **СОЗДАНИЕ НОВОГО ПРОЕКТА**

Запустите AnyLogic. Закройте все ранее открытые проекты, для этого выберите в основном меню **Файл** / **Закрыть все**.

Для построения нового проекта щелкните кнопку **Создать** панели инструментов, либо выберите в основном меню **Файл** / **Создать** / **Модель**.

В появившемся диалоговом окне установите рабочую папку: рабочий_диск:\номер_группы\ФИОстудента, наберите `heart` как имя модели и щелкните **Далее**. В следующем окне оставьте выбранным по умолчанию **Начать создание модели с** «**нуля**» и щелкните **Готово**. Новый проект под названием `heart` будет создан.

Окно редактора нового проекта содержит несколько панелей. Слева в панели проектов автоматически строится дерево проекта. Оно обеспечивает навигацию по всем элементам проекта, которые будут создаваться при построении модели. Для нашего проекта в нем уже создан корневой класс активного объекта с именем `Main`, а для проведения экспериментов с моделью уже создан один эксперимент с именем `Simulation`. Центральная панель – панель графического редактора класса активного объекта в которой мы будем производить основные построения модели.

Нижняя панель – это панель свойств выделенного элемента модели. В данном случае панель показывает свойства класса корневого объекта с именем `Main`, если окно его структуры активно. Измените имя корневого объекта нашей модели, назвав его `Heart` (вместо установленного по умолчанию имени `Main`). Для этого в поле **Имя** вкладки **Основные** панели **Свойства** корневого объекта введите `Heart` вместо `Main`. В панели проектов имя корневого объекта сразу изменится.

Поскольку AnyLogic создает для каждого класса активного объекта соответствующий Java класс, при задании имени класса активного объекта нужно руководствоваться правилами названия классов в Java. Пожалуйста, начинайте имя класса с заглавной буквы.

1.2. **ПОСТАНОВКА ЗАДАЧИ**

Мы рассмотрим простейшую математическую модель, описывающую процессы, похожие на биение сердца. Эта модель описана двумя дифференциальными уравнениями первого порядка:

$$\frac{dx}{dt} = \frac{(x - x^3 - b)}{\varepsilon}; \qquad \frac{db}{dt} = x - x_0,$$

где: `x` – радиус сердца, `x0` – его начальное значение, `b` – переменная, а ε– параметр `eps`.

В этой модели мы исследуем характер зависимостей переменных`x` и `b` от времени при разных значениях параметра `eps`, а также построим фазовую диаграмму зависимости радиуса `x` от переменной `b`.

1.3. **ПОСТРОЕНИЕ МОДЕЛИ**

В нашей модели должны присутствовать две переменные состояния – `x` и `b`, и два параметра – `x0` и `eps`, где `x0` - начальное значение `x`. Начальное значение переменной `b` зададим константой.

В модели переменная `x` определяется дифференциальным уравнением:

```
dx/dt = (x - x^3 - b)/eps
```

с начальным значением `x`, равным `x0`. В AnyLogic можно подобные зависимости задавать именно в таком аналитическом виде. Для того чтобы таким образом определить переменную `x`, зададим ее в форме накопителя.

Для этого перетащите мышью элемент **Накопитель**, расположенный во вкладке **Системная динамика** панели **Палитра** на диаграмму класса активного объекта`Heart`. Пиктограмма появится в поле редактора с именем `stock`. Одновременно внизу вместо окна свойств объекта `Heart` появится окно свойств выделенного накопителя. В это окно в

поле имени **Имя** вместо предопределенного имени `stock` введите `x` (рис. 2.2) и нажмите клавишу <Enter>.

В поле **Начальное значение** введите `x0`, после чего, см. рис. 2.2, определим формулу для вычисления `x` в поле dx/dt = следующим образом:

```
(x - x*x*x - b) /eps
```

Если Вы работаете с программой AnyLogic версии 6.6 и выше, то поле **Начальное значение** будет недоступным. Для того чтобы ввести в это поле формулу, нужно выбрать **Режим задания уравнения** `Произвольный`, см. рис. 2.1.

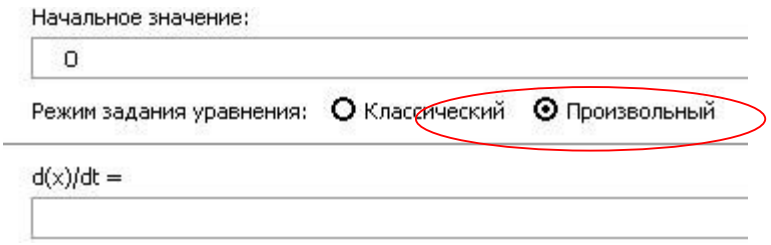


Рис. 2.1

При выделенной пиктограмме накопителя ☐ его имя можно перемещать по диаграмме класса. Саму пиктограмму накопителя также можно перемещать при нажатой на ней левой кнопке мыши.

Вторая переменная `b` задана дифференциальным уравнением `db/dt = x - x0`. Ее создадим в модели аналогичным образом. Установим начальное значение `b` равное 0. В поле **Начальное значение** окна свойств переменной `b` величину `0` можно не записывать: если это поле пусто, по умолчанию значение переменной считается нулевым.

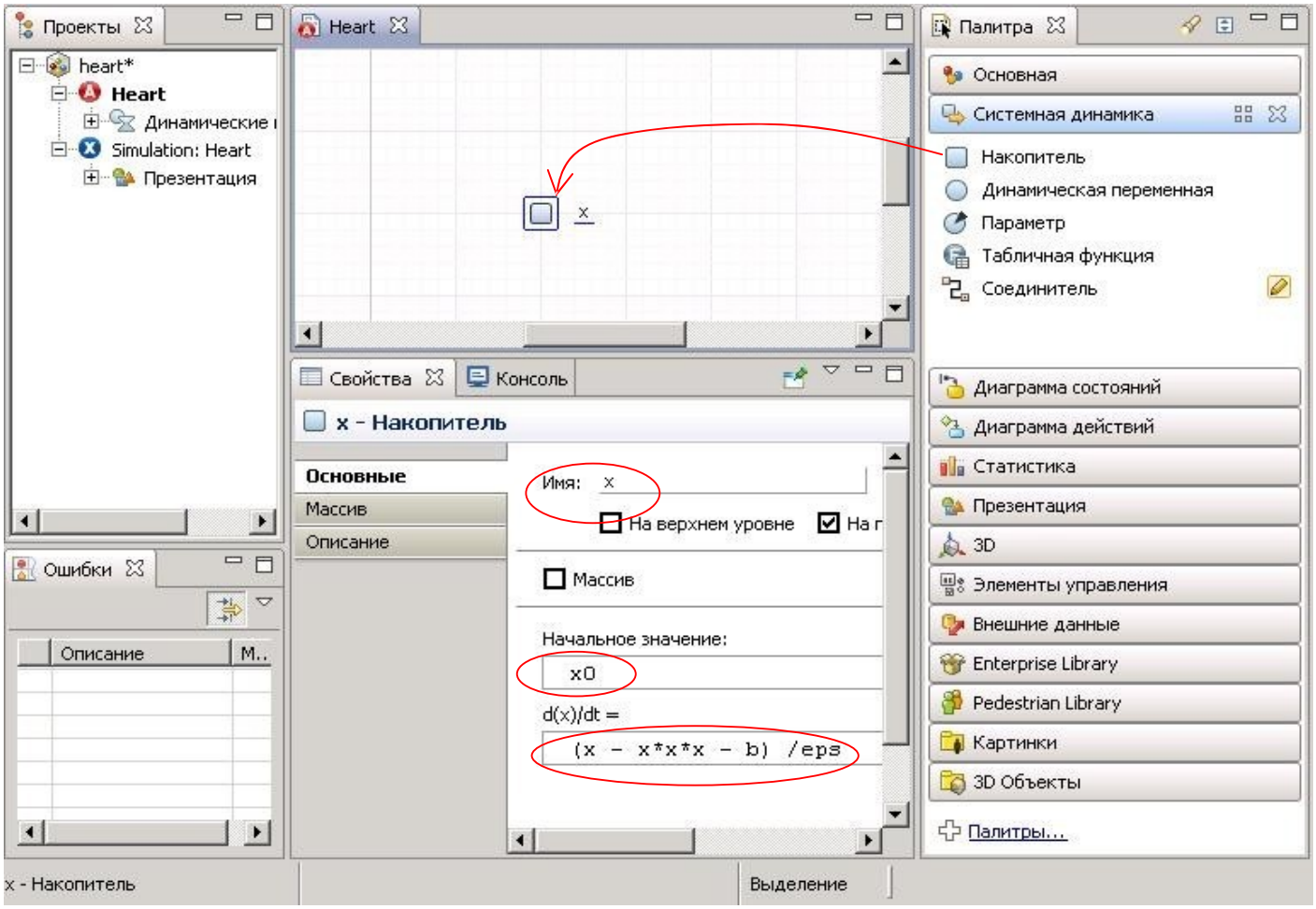


Рис. 2.2

На диаграмму класса активного объекта можно помещать текстовые комментарии. Для этого перетащите элемент **Текст** со вкладки **Презентация** панели палитр на диаграмму класса, рядом с накопителями. Введите следующий комментарий: `Структура активного объекта`. Его можно редактировать в поле **Текст** окна свойств этого комментария.

Для проверки правильности синтаксиса (модели можно использовать кнопку **Построить модель** панели инструментов или нажать клавишу F7. Если щелкнуть на этой кнопке, то выполнится компиляция разрабатываемой модели в программный код на языке Java. Щелкните по кнопке **Построить модель**. В нашем примере обнаружились ошибки (рис. 2.3): действительно, нами не определены параметры `x0` и `eps`.

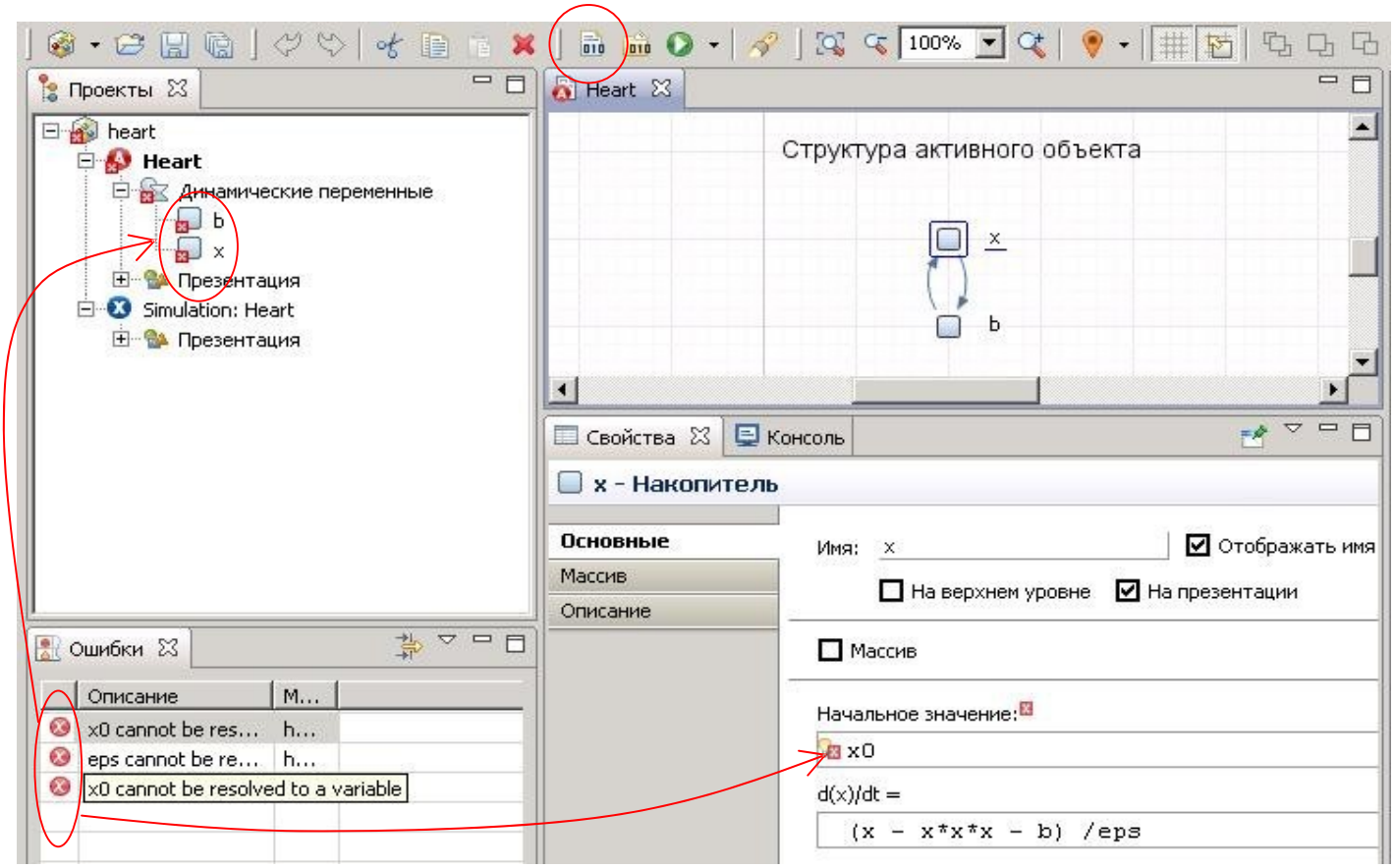


Рис. 2.3

На наличие ошибки указывает появившиеся символы [X] в панели **Ошибки**. Для каждой ошибки показывается ее описание и местоположение - имя элемента модели, при задании которого эта ошибка была допущена. Двойной щелчок мышью по ошибке, в зависимости от того, где она была совершена, приведет к открытию того или иного редактора или панели. Если, например, это графическая ошибка, то будет открыт графический редактор, отображающий диаграмму соответствующего класса активного объекта, в котором будут выделены фигуры, которые были неправильно нарисованы.

Для завершения построения модели указанные пропущенные параметры нужно задать. Пусть `x0 = 0.5, eps = 0.0`1Перетащите мышью пиктограмму **Параметр** с вкладки **Основная** панели **Палитры** на диаграмму класса активного объекта `Heart`. Назовите его `x0`, затем на вкладке **Основные** панели свойств этого параметра введите в поле **Значение по умолчанию** – 0.5. Остальные поля оставьте без изменения. Переменная `eps` со значением 0.01 задается так же. Снова выполните проверку синтаксиса: нажмите кнопку **Построить модель**. В результате на экране вы получите следующее - рис. 2.4.

Надпись «Построение удачно завершено» в левом нижнем углу окна программы свидетельствует о том, что синтаксис модели правильный.

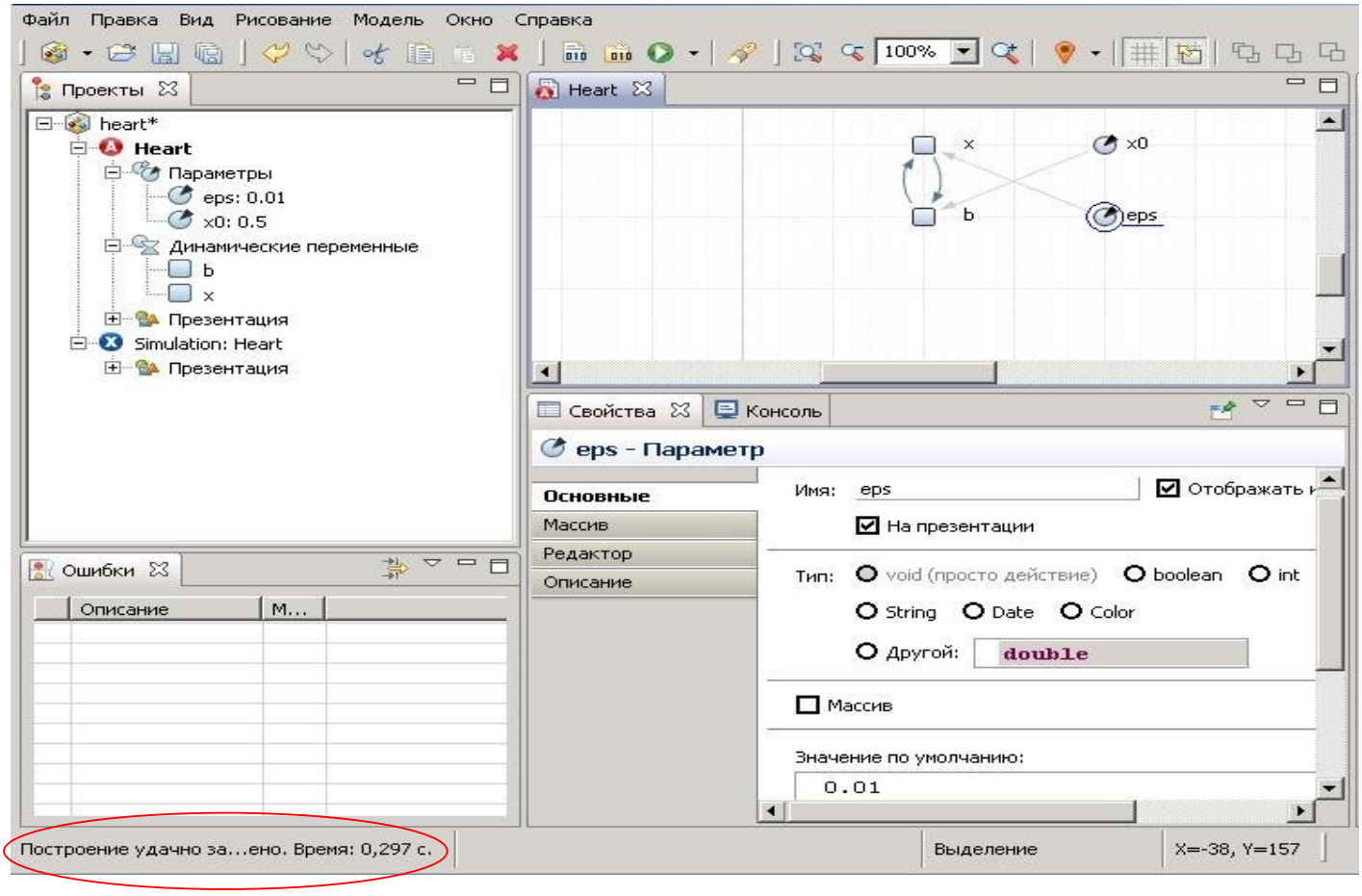


Рис. 2.4

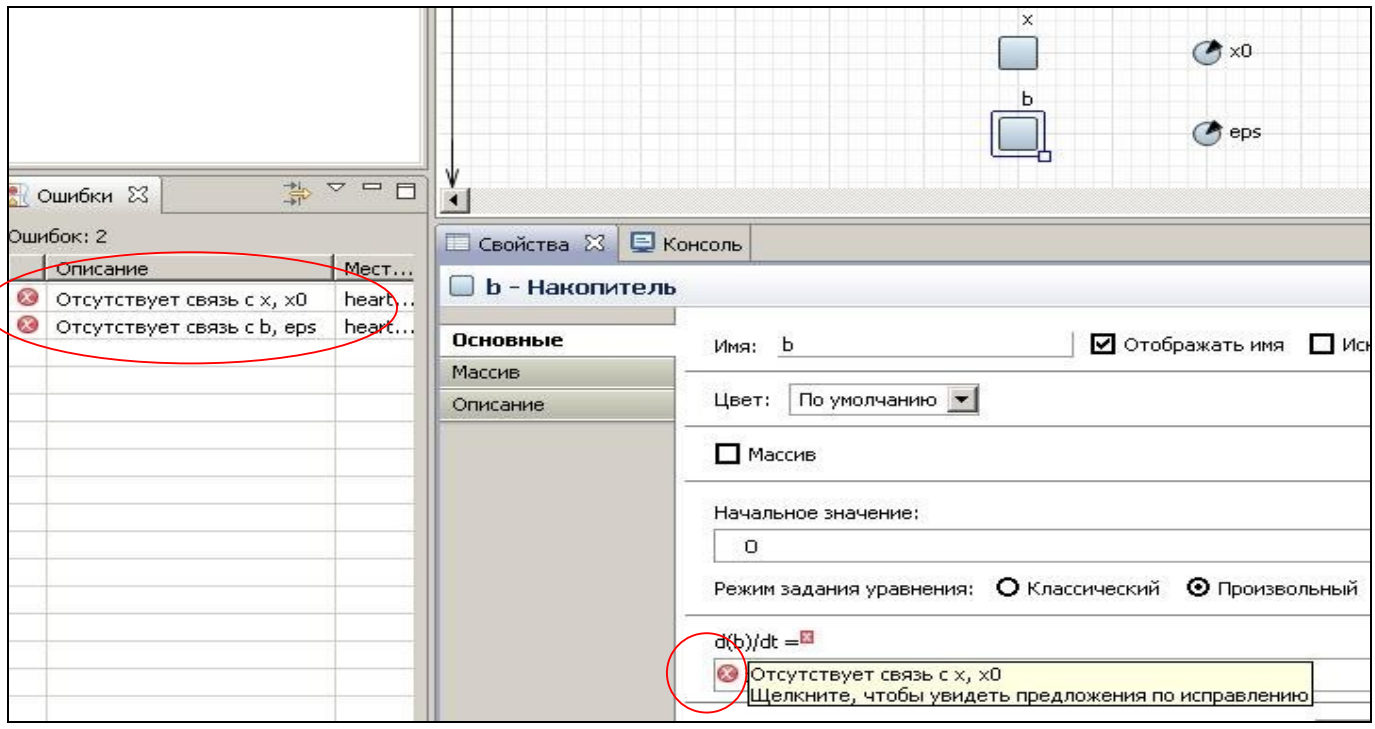


Рис. 2.5

В версии 8 программы AnyLogic связи между переменными автоматически не построятся, и в панели **Ошибки** мы увидим, что две ошибки остались неисправленными, а

в их описании будет указано, что: `Отсутствует связь с …`, см. рис.3.5.

Для задания связи между переменными выделите мышью переменную, например `b`, и в панели свойств установите курсор в поле с уравнением `db/dt = x - x0`Затем щелкните мышью по красному крестику слева от уравнения, откроется всплывающее окно с предложением создать связь, рис. 2.6.

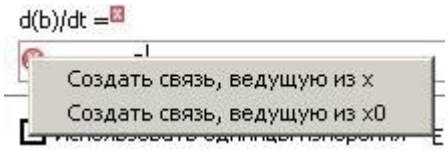
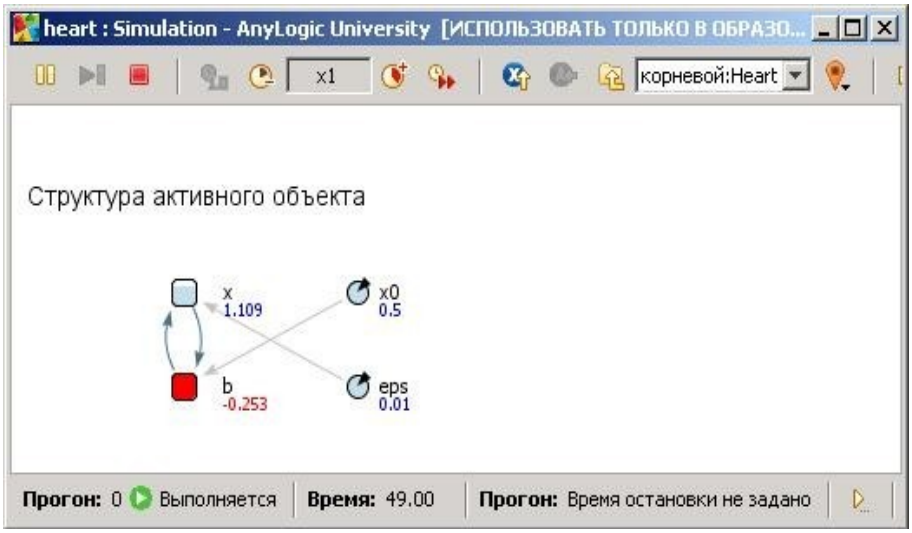


Рис. 2.6

Создайте все связи и выполните проверку синтаксиса, если все правильно, то появится сообщение об удачном построении модели.

1.4. **ЗАПУСК МОДЕЛИ**



Щелкнув на кнопке запуска модели, после компиляции откроется окно презентации эксперимента. Запустив эксперимент, увидим структуру активного объекта: переменные и параметры с их значениями (рис. 2.7). Переменные `b` и `x` в этом окне начнут изменяться в соответствии с определенными для них уравнениями.

Рис. 2.7

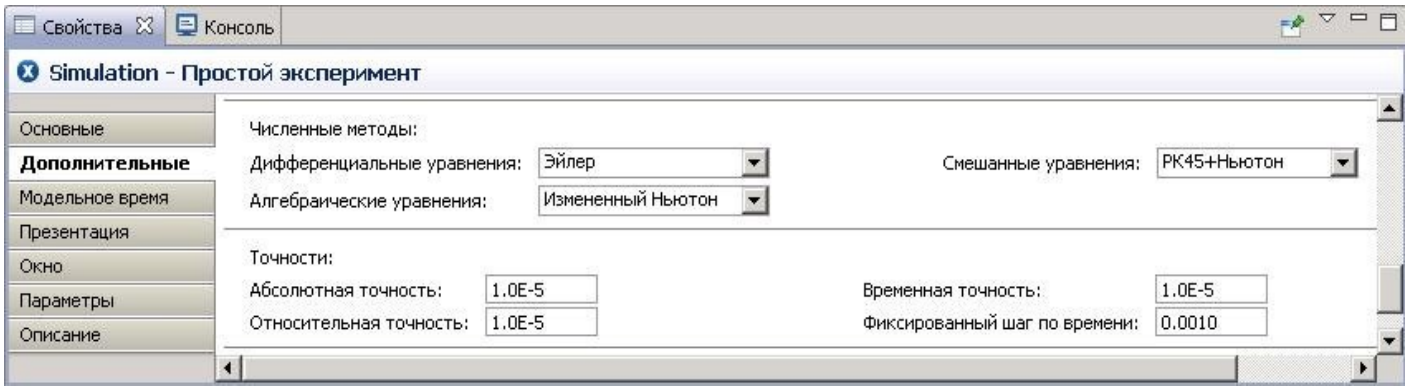
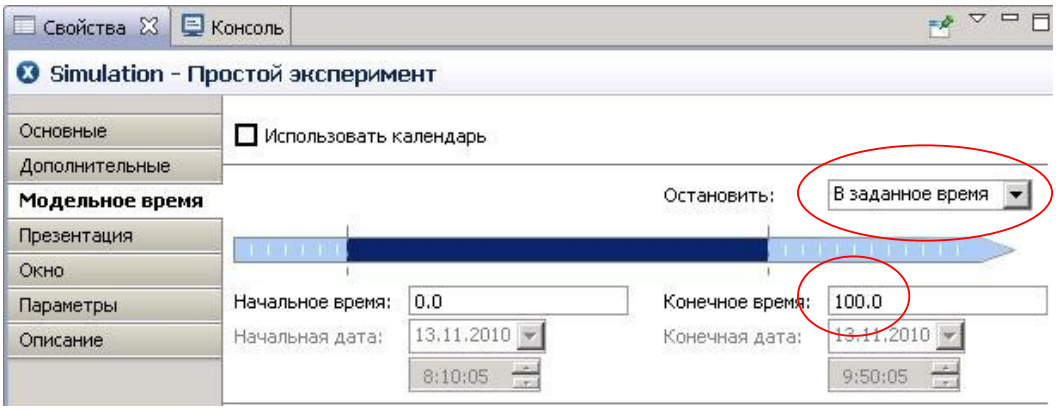


Рис. 2.8

Системы дифференциальных и алгебраических уравнений, при выполнении модели, решаются одним из встроенных численных методов. Сам метод и необходимая точность решения выбираются автоматически, но можно изменить предварительные установки во вкладке **Дополнительные** панели свойств объекта `Simulation`, рис. 2.8.

По умолчанию модель будет выполняться бесконечно, но можно задать условие остановки эксперимента во вкладке **Модельное время** панели свойств объекта `Simulation`, рис. 2.9.



Проведите несколько экспериментов с различными скоростями выполнения данной модели, используя кнопки останова и запуска.

Рис. 2.9

1.5. **ГРАФИКИ И ДИАГРАММЫ**

AnyLogic позволяет наглядно представить поведение модели, в частности, представить изменения во времени всех ее переменных. Существуют два способа просмотра графиков: с помощью окон инспекта и с помощью диаграмм.

Окна инспекта мы подробно рассмотрели в работе №1, здесь отметим лишь то, что эти графики строятся на базе автоматически создаваемых наборов данных, в которые периодически записываются новые и новые значения переменных. Если кривая Вашего графика покажется Вам не гладкой, а "рваной", то причина этого может заключаться в том, что AnyLogic недостаточно часто обновляет наборы данных новыми значениями.

Чтобы изменить частоту обновления автоматически создаваемых для переменных наборов данных перейдите на страницу свойств **Дополнительные** того класса активного объекта, на диаграмме которого находится переменная и измените период обновления данных в поле **Период**, рис. 2.10.

Для получения полнофункциональных графиков лучше воспользоваться диаграммами AnyLogic, которые позволяют динамически визуализировать данные, собираемые в результате работы модели. Набор диаграмм схож с тем, что предлагается программой MS Excel.

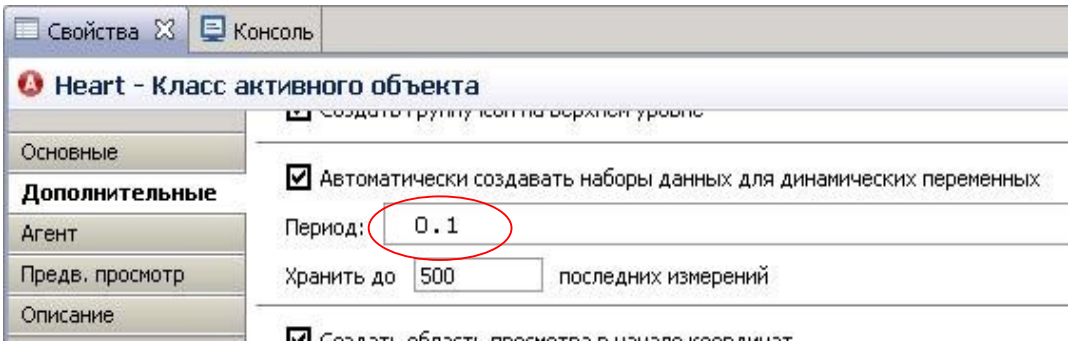


Рис. 2.10

Построим графики зависимостей переменных `b` и `x` от времени и фазовую диаграмму `b` от `x`. Перетащите элемент **График** из палитры **Статистика** в то место графического редактора, где Вы хотите нарисовать график. Перейдите на страницу **Основные** панели **Свойства**.

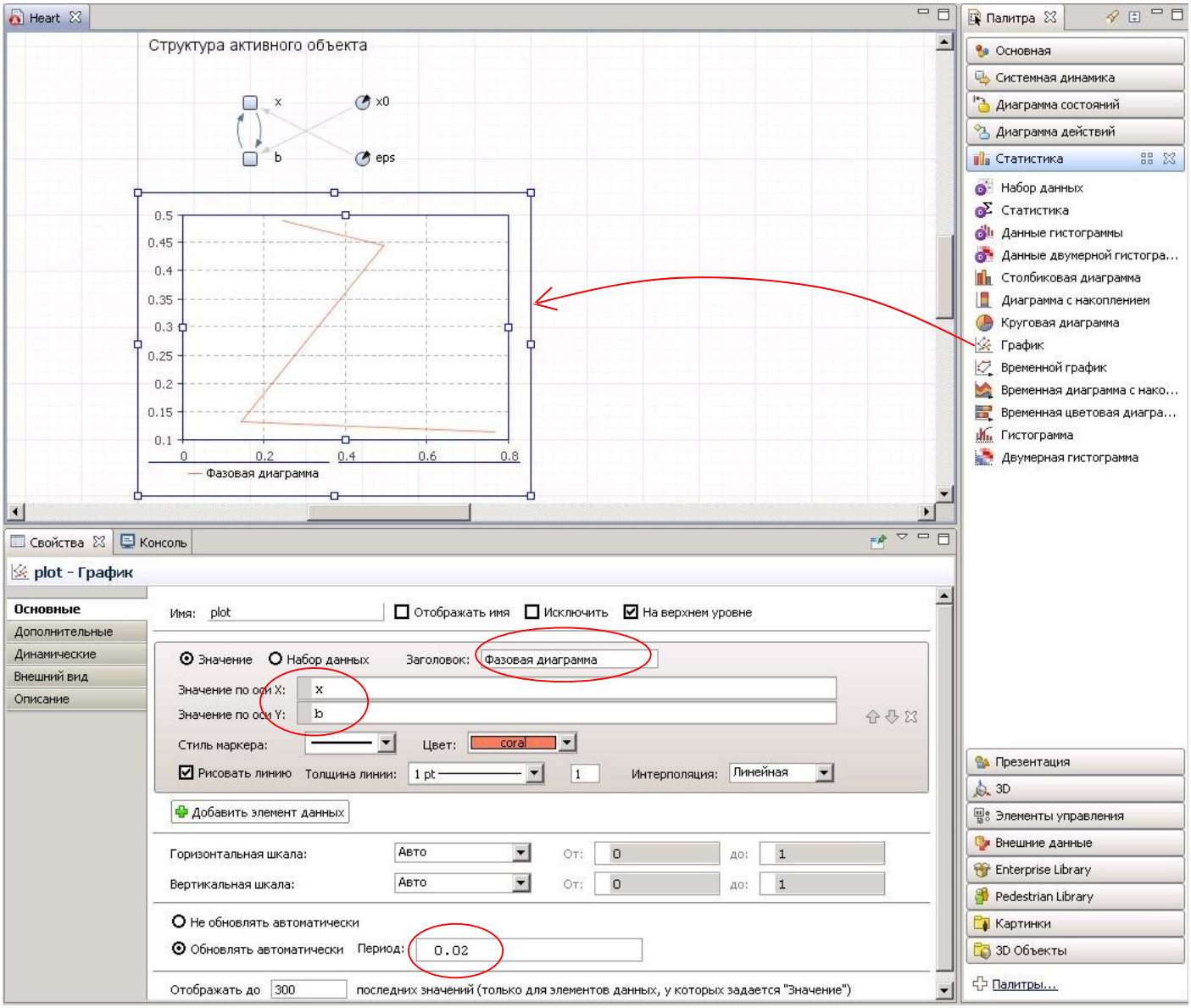


Рис. 2.11

Щелкните мышью по кнопке **Добавить элемент данных**. При этом над кнопкой появится новая секция свойств, задающая настройки нового элемента данных, который будет отображаться на данном графике. В поле **Значение по оси** X введите имя переменой X, а в поле **Значение по оси** Y – имя переменной b. Задайте заголовок `Фазовая диаграмма` для этого элемента данных в поле **Заголовок**. Для того чтобы получить гладкую диаграмму нужно уменьшить период обновления данных в поле **Период**, рис. 2.11.



Рис. 2.12

Аналогично постройте временн ой график для переменных `x` и `b`, разместив их на одном графике. **Временной диапазон** укажите равным 10, рис.2.13.

AnyLogic позволяет настраивать внешний вид и функциональность диаграмм. Поэкспериментируйте с установкой цвета графиков, опциями отображения наборов данных, размерами отображаемого окна, прозрачностью, легендой, и т. п.

Проведите серию экспериментов с моделью, перезапуская ее с различными параметрами.

Для изменения параметров в процессе выполнения модели, нужно в окне инспекта этого параметра щелкнуть мышью по кнопке с карандашом в верхней части окна, рис. 2.12.

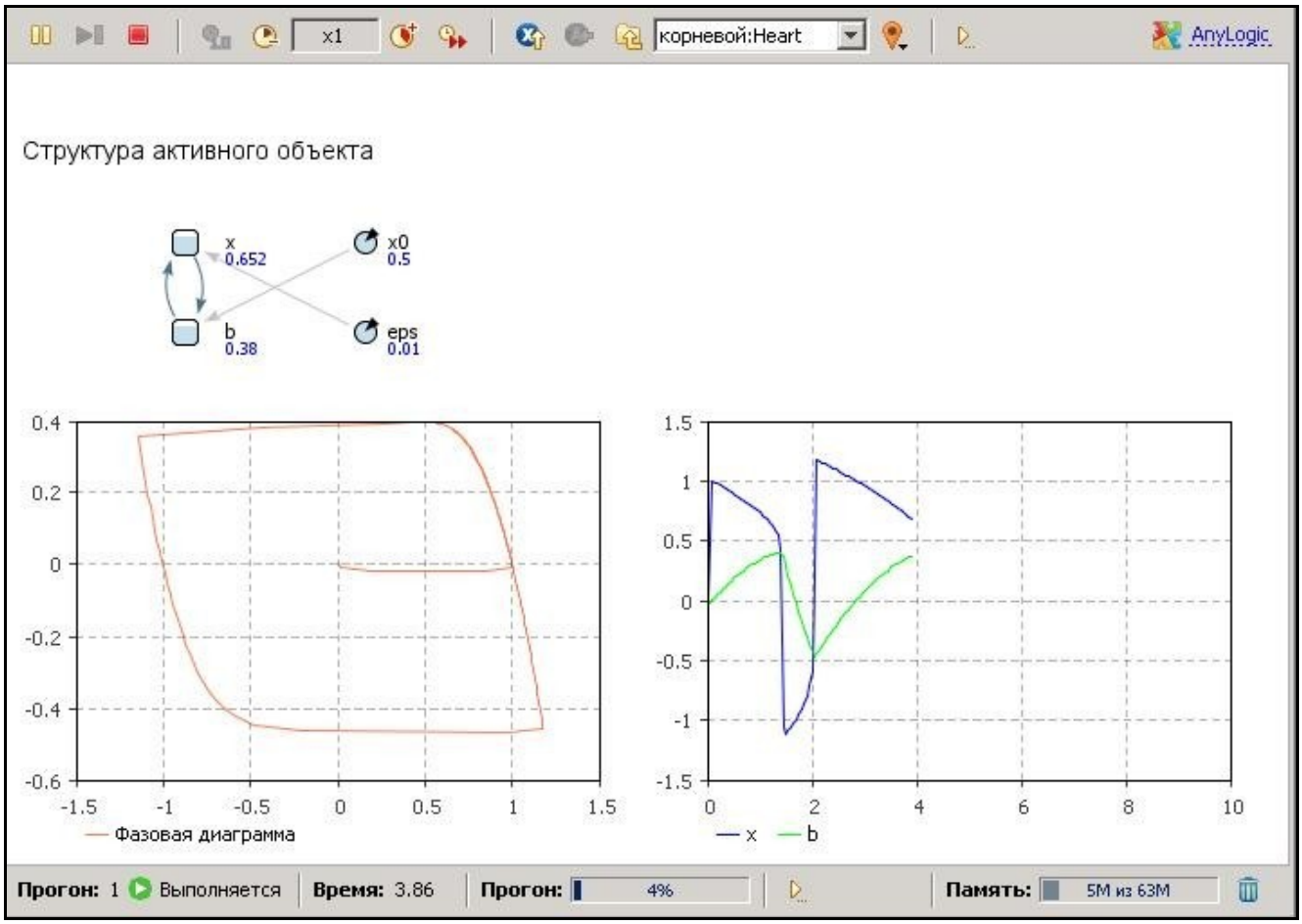


Рис. 2.13

1.6. **ПРЕЗЕНТАЦИЯ МОДЕЛИ**

Для лучшего понимания динамики модели и наблюдения за процессами, в AnyLogic можно строить анимированные изображения, состоящие из динамических элементов. Графические элементы называются динамическими, поскольку все их параметры - координаты, размер, цвет и даже их видимость – в процессе выполнения модели можно сделать зависимыми от переменных и параметров, которые меняются со временем при выполнении модели.

Остановите выполнение модели и вернитесь на диаграмму класса `Heart`. Прямо на диаграмме мы будем рисовать презентацию сердца.

1.6.1. **ОБЪЕКТЫ В ПРЕЗЕНТАЦИИ**

Построим презентацию сердца в виде изображения овала, радиус которого будет меняться. Этот радиус является функцией от значения переменной `x` модели. Для построения изображения овала перетащите мышью из вкладки **Презентация** панели **Палитра** пиктограмму **Овал** на диаграмму класса. Внизу появится окно свойств этого овала. По умолчанию имя этого объекта будет `oval`, координаты `X` и `Y` соответствуют месту, куда мы поместили овал, а радиус X и радиус Y соответствуют тому, что мы нарисовали.

1.6.2. ДИНАМИЧЕСКИЕ ПАРАМЕТРЫ ГРАФИЧЕСКИХ ОБЪЕКТОВ

Запустив модель, мы увидим, неподвижный овал, находящийся в заданном месте. В AnyLogic принята следующая концепция: каждая характеристика графического элемента имеет два значения: статическое и динамическое. Статическое значение определяет параметр (координату, угол поворота, цвет и т. п.) объекта как константу. Динамическое определяет значение этого параметра в процессе выполнения модели и может быть определено как значение любой переменной модели. Поэтому у графических объектов в панели свойств имеются вкладки **Основные** и **Дополнительные** для статических значений, и вкладка **Динамические** – для динамических значений параметров. Если динамическое значение не определено, графический объект сохранит свое статическое значение.

Выделите овал, представляющий динамику сердца. Статические значения его параметров задайте так: в панели свойств овала на вкладке **Основные** цвет заливки определите бордовым, цвет линии границы - красным; толщину линии границы установите 2.

Изменение объема сердца представим радиусами овала на вкладке **Динамические** как функцию от переменной `x`, как показано на рис. 2.14.

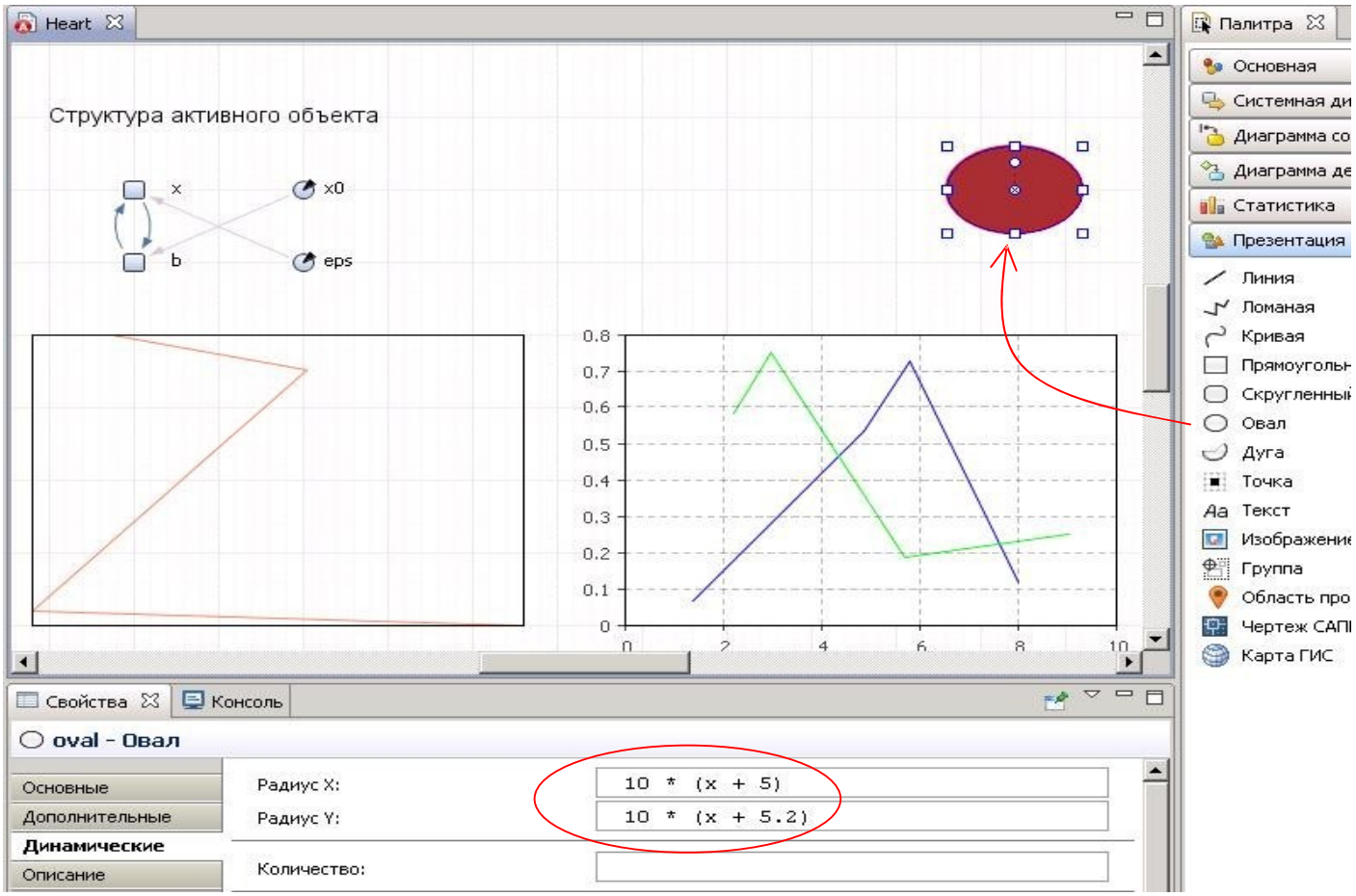


Рис. 2.14

Запустите модель. Проведите эксперименты с установкой различных параметров модели (`x0` и `eps`) и наблюдайте, как изменяется характер сердцебиения.

1.6.3. **СЛАЙДЕРЫ В ПРЕЗЕНТАЦИИ**

В AnyLogic существует элемент управления, позволяющий пользователю графически выбирать число из заданного диапазона значений путем перетаскивания рукоятки – бегунок.

Бегунки или их еще называют слайдеры, обычно используются для изменения значений численных переменных и параметров во время выполнения модели.

Перетащите мышью с вкладки **Элементы управления** панели **Палитра** пиктограмму **Бегунок** на диаграмму класса `Heart` между структурой и анимационным овалом. В поле **Связать с** панели свойств введите имя параметра `x0`, а минимальное и максимальное значения, которые можно регулировать слайдером, установите 0 и 1. Добавить подписи к слайдеру можно простым нажатием кнопки **Добавить метки**, рис. 2.15. Запустите модель и проверьте действие слайдера.

Аналогично сделайте слайдер для параметра `eps`, установив ограничения от 0,01 до 0,5.

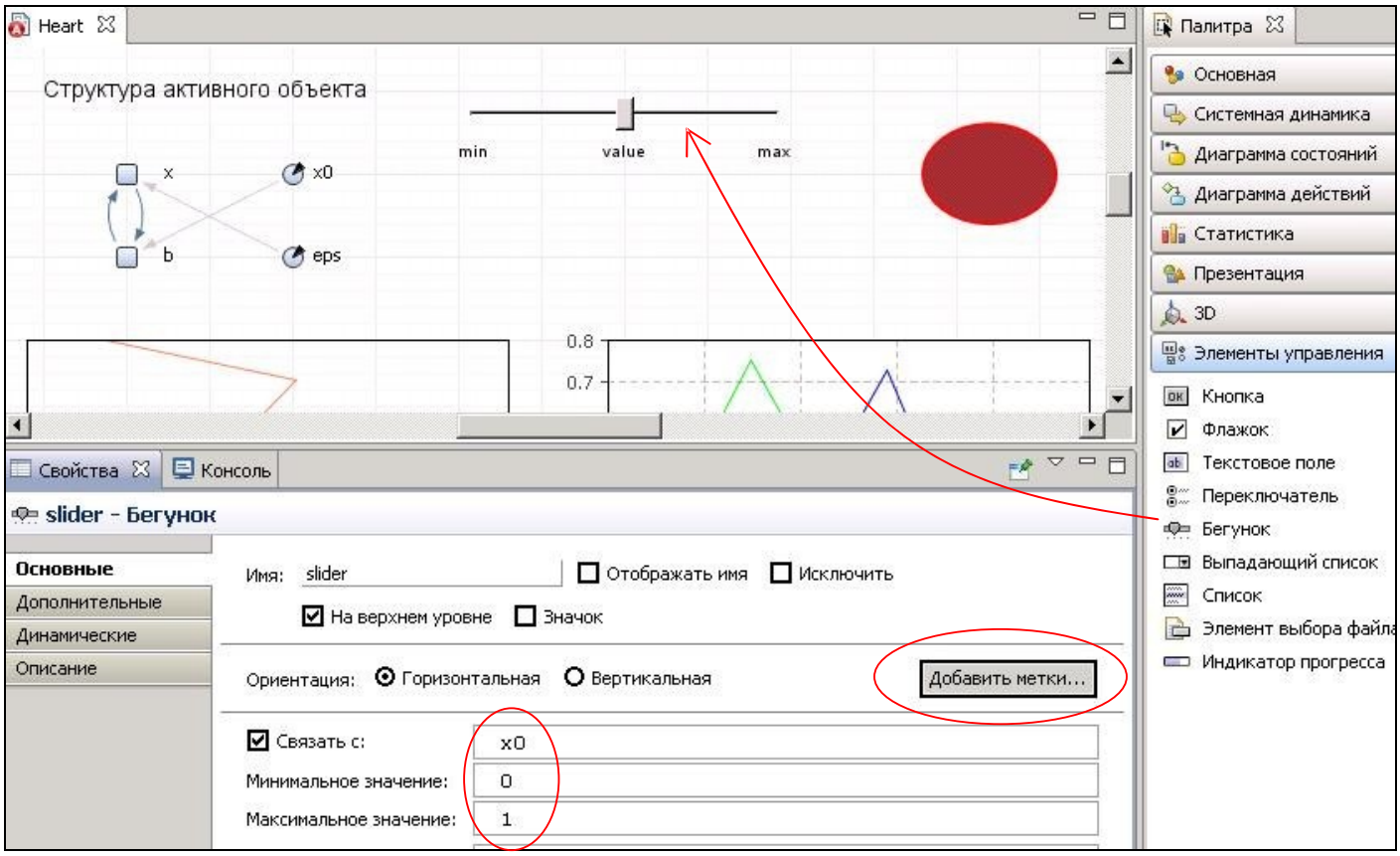


Рис. 2.15

1.6.4. **ТЕКСТ В ПРЕЗЕНТАЦИИ**

В верхнюю часть диаграммы класса `Heart` введите поясняющий текст. Для этого перетащите мышью пиктограмму **Текст** с вкладки **Презентация** панели **Палитра**. В поле **Текст** вкладки **Основные** введите текст `Модель сердца`, выберите шрифт Century Gothic, стиль **Полужирный**, размер 22. На вкладке **Дополнительные** установите координаты (X, Y) этого объекта (20, 10).

Следующий текст, содержащий пояснение к модели, введите чуть ниже и правее: `Это упрощенная модель динамики сердечных сокращений. Модель описывается двумя дифференциальными уравнениями: dx/dt=(x-x^3-b)/eps и db/dt=x-x0. Модель позволяет изменять параметры и наблюдать изменение ритма и формы пульсаций. При значении x0 = 0.6 сердце останавливается.`
Установите выравнивание по правому краю, все другие параметры текста можете оставить по умолчанию. Для того чтобы текст уместился по ширине, в нужных местах следует вставить перевод строки.

Элемент **Текст** также имеет динамические свойства, как и другие графические элементы. Это значит, что можно в процессе выполнения модели динамически изменять положение и ориентацию текста, его цвет, и даже сам текст.

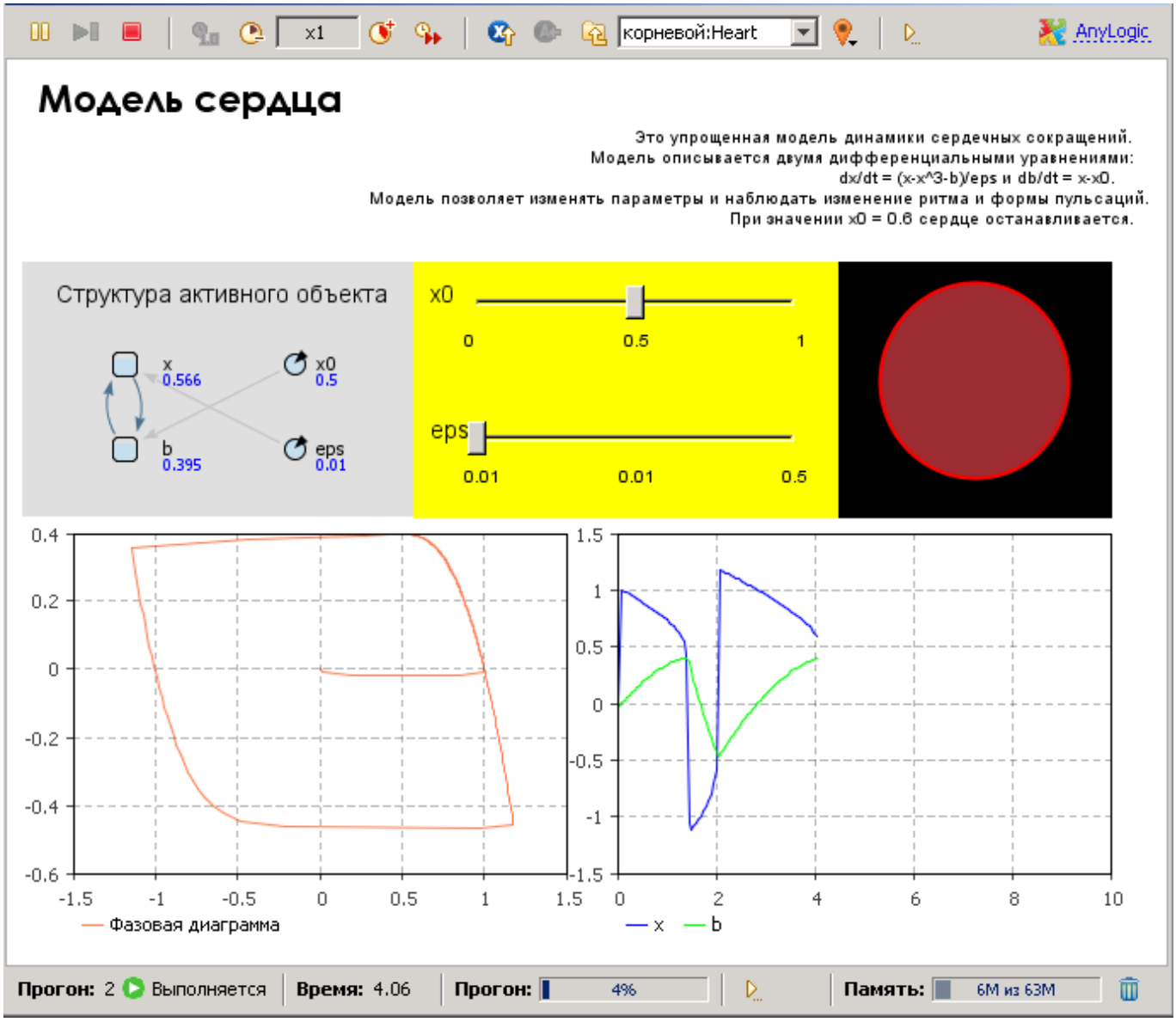


Рис. 2.16

Введите в поле презентации прямоугольники, как показано на рис. 2.16, и поместите их на задний план, чтобы они выделяли функциональные блоки построенной модели. Для того чтобы фигура была фоном и не закрывала другие изображения, в контекстном меню данного прямоугольника выполните команду **Порядок** / **На задний план**.

Сделайте подписи к слайдерам, чтобы было понятно, какими параметрами мы управляем.

ЗАДАНИЕ 2

СТЕЙТЧАРТЫ: МОДЕЛЬ ПЕШЕХОДНОГО ПЕРЕХОДА

2.1. ОПИСАНИЕ ПРОБЛЕМЫ

Светофор, регулирующий движение автомобилей на пешеходном переходе, может находиться в следующих состояниях: движение транспорта разрешено (зеленый), приготовиться к запрещающему сигналу (мигающий зеленый), приготовиться к остановке (желтый), движение запрещено (красный) и приготовиться к движению (красный и желтый).

Светофор работает в автоматическом режиме. В каждом состоянии светофор находится определенный постоянный период времени.

2.2. **ПОСТРОЕНИЕ МОДЕЛИ**

Создайте новый проект под названием `Svetofor` и назовите класс корневого активного объекта `Model`.

Наша модель будет иметь только один активный объект, представляющий светофор, поэтому корневой объект `Model` будет единственным активным объектом нашей модели. На диаграмму класса активного объекта `Model` поместите **Начало диаграммы состояний** из панели **Диаграмма состояний** и назовите ее `Для_автомобилей`, заметьте, что AnyLogic может работать с элементами, набранными кириллицей. Перетащите мышью элемент **Состояние** под стрелочку начала диаграммы как показано на рис. 3.1.

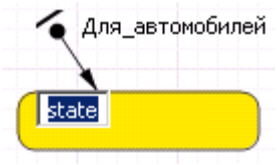


Рис. 3.1

Имя состояния, как и все другие его параметры, можно редактировать в окне его свойств.

Для того чтобы построить стейтчарт, следует использовать элементы из палитры **Диаграмма состояний**, рис. 3.2.

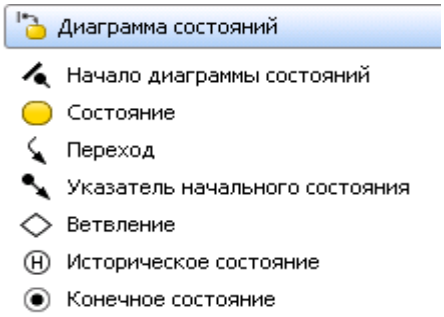


Рис. 3.2

1. определяет начальное состояние всего стейтчарта,
2. с помощью кнопки рисуются состояния (как простые, так и гиперсостояния),
3. используется для рисования переходов между состояниями,
4. определяет начальное состояние внутри сложного состояния,
5. используется для рисования состояния, являющегося "финальным" в поведении активного объекта.

Заметьте, что для любого выделенного объекта внизу появляется панель его свойств, в котором можно изменить параметры и, в частности, имя объекта, если это необходимо. Структурные ошибки при рисовании стейтчарта - повисшие переходы, дублированные указатели начального состояния и т. п. - выделяются в панели **Проекты** значком X красного цвета и записью в панели **Ошибки**. Выделенные переходы должны иметь на концах зеленые точки, если точки белые, это значит, что переход не соединен с состоянием – висит.

В соответствии с алгоритмом работы светофора помимо начального состояния в модель нужно ввести дополнительные состояния (рис. 3.3). Начальное состояние назовите `движение` – движение автомобилям разрешено (зеленый свет), затем светофор переходит в состояние `внимание` – внимание (мигающий зеленый), `медленно` – приготовиться к остановке (желтый свет), остановка транспорта `stop` – запрет движения (красный свет) и `приготовиться` – приготовиться к движению (красный и желтый свет горят одновременно).

Состояние `внимание` представим гиперсостоянием с парой переключающихся элементарных состояний: в одном из них зеленый горит (состояние `A`), в другом – нет (состояние `B`). Для построения гиперсостояния сначала создайте обычное состояние, увеличьте его (растянув мышью) и поместите внутрь другое состояние. Постройте эти

состояния и соедините их переходами, как показано на рис. 3.3.

Зададим условия срабатывания переходов. Переходы в нашем автоматическом светофоре выполняются по таймауту, т. е. по истечении интервала времени, который прошел с момента прихода системы в данное состояние.



1. В состоянии `движение` светофор находится 10 секунд,
2. затем 7 секунд зеленый сигнал мигает,
3. в состоянии `медленно` 4 секунды горит желтый,
4. в течение 10 секунд движение запрещено и
5. 4 секунды светофор находится в состоянии `приготовиться`.

В нашей модели единица модельного времени соответствует 1 секунде реального времени.

Для задания условий срабатывания переходов, выделите переход `t1`, и в поле **Происходит** оставьте без изменения вариант **По таймауту**, а в поле **По таймауту** введите 10 (рис. 3.4).

Аналогично задайте условия срабатывания других переходов. Между состояниями `A` и `B` переходы должны срабатывать через 1 секунду.

Рис. 3.3

Запустите модель. Активное в данный момент состояние подсвечивается красным. Переход, ожидающий истечения таймаута подсвечивается синим. Проведите эксперименты с моделью при различных масштабах времени.

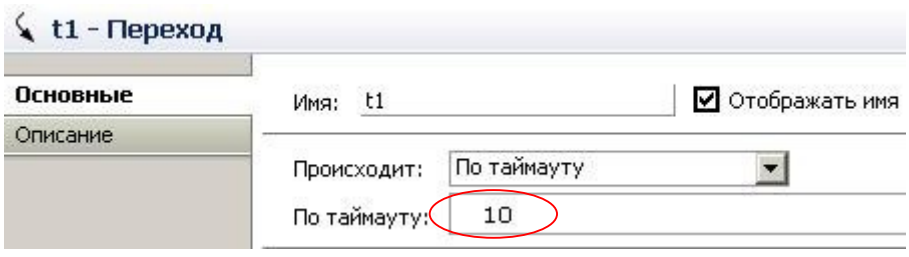
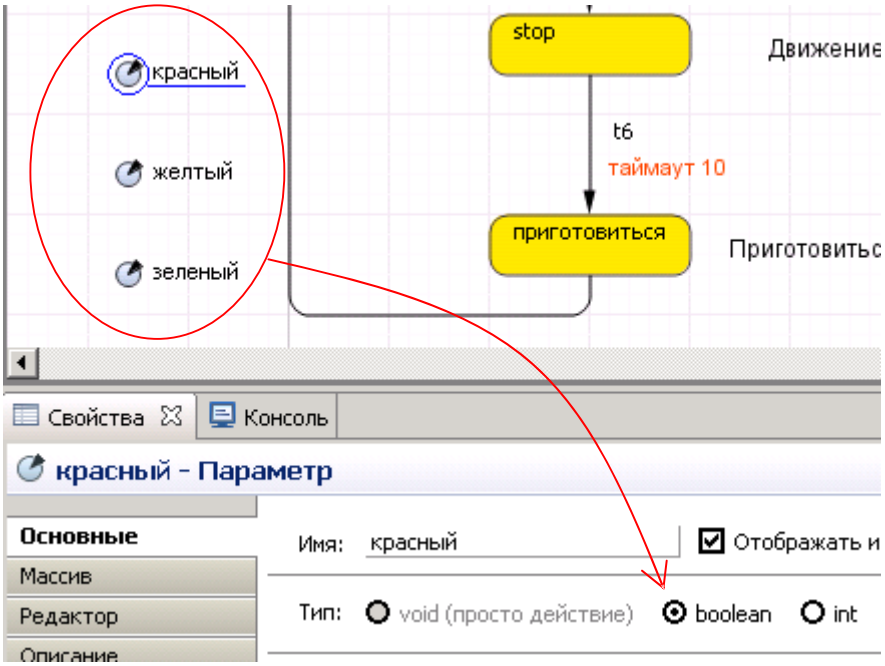


Рис. 3.4

В каждом состоянии светофора должен гореть определенный сигнал: в состоянии `движение` должен гореть зеленый, в состоянии `приготовиться` должны гореть красный и желтый одновременно и т. п. Перейдите на диаграмму класса активного объекта `Model`. Создайте три параметра логического типа: `красный`, `желтый` и `зеленый`, которые будут

принимать истинное значение тогда, когда у светофора горит соответствующий сигнал: красный, желтый или зеленый (рис. 3.5). Начальные значения этих булевых параметров можно не задавать: по умолчанию они будут равны `false`.



Мы создали стейтчарт именно для управления значениями этих параметров, каждое состояние отвечает за зажигание своего света или их комбинации. Например, в состоянии `медленно` должен гореть желтый, в состоянии `stop` должен загореться красный свет, а в состоянии `приготовиться` должны гореть красный и желтый одновременно. Запрограммируем эти действия.

Рис. 3.5

1. В свойствах состояния `движение` в поле **Действие при входе** запишите `зеленый=true;`, а в поле **Действие при выходе** запишите `зеленый = false;` (рис. 3.6).
2. То же самое запишите для состояния `B` гиперсостояния `внимание`, а у состояния `A` эти поля нужно оставить пустыми – когда светофор находится в этом состоянии, он не горит.
3. Аналогично, в состоянии `медленно` нужно включить желтый сигнал, т. е. при входе в это состояние установить параметр `желтый` в `true`, а при выходе из этого состояния установить его в `false`.
4. Для состояния `stop` аналогично опишите состояния параметра `красный`,
5. а для состояния `приготовиться` оба параметра `красный` и `желтый` нужно установить в `true` при входе, и в `false` при выходе.

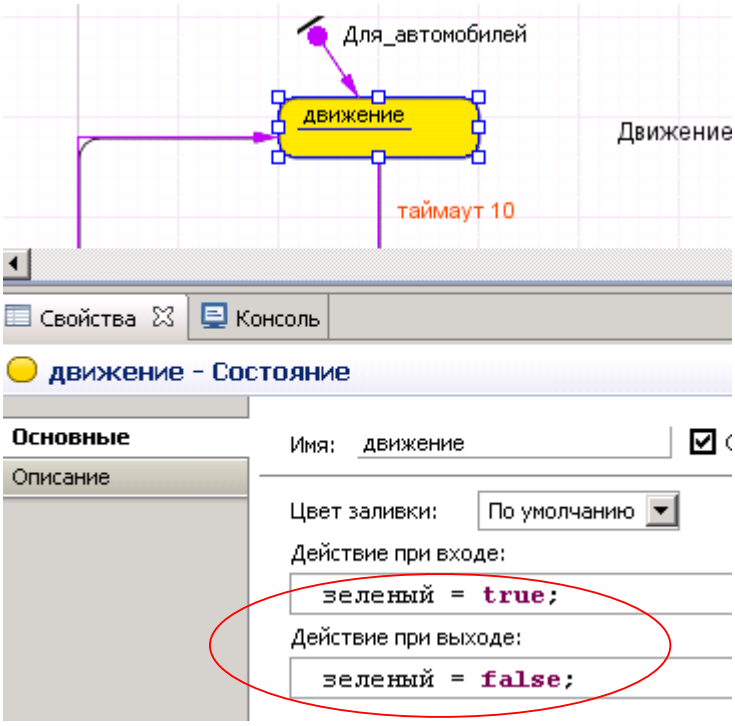


Рис. 3.6

Запустите модель на выполнение. Вы увидите, что параметры `зеленый`, `желтый` и

`красный` будут переключаться между значениями `истина` и `ложь` в соответствии с алгоритмом переключения светофора.

2.3. **ПРЕЗЕНТАЦИЯ МОДЕЛИ**

Презентация модели рисуется в той же диаграмме (в графическом редакторе), в которой задается и диаграмма моделируемою процесса, (рис. 3.7). Графические объекты цвета сигналов светофора в презентации имеют динамические параметры, все остальные – статические. Проезжую часть удобно нарисовать с помощью графических примитивов типа **Прямоугольник**, а светофор с помощью примитива - **Ломаная**. Сигнальные элементы светофора строятся из трех овалов, повернутых на 45 градусов (поле **Поворот** вкладки **Дополнительные** окна свойств овала).

Установим динамическое значение цвета верхнего сигнала светофора: если переменная `красный` истинна, то цвет должен быть `red` (красный), в противном случае его цвет нужно установить `gray` (серый). Это записывается следующим условным выражением на языке Java:

```
красный? red: gray
```

Цвет среднего и нижнего овалов, следует установить в поле их динамических значений соответственно так:

```
желтый? yellow: gray
```

```
зеленый? green: gray
```

red, yellow, green и gray – предопределенные константы, обозначающие стандартные цвета.

Запустите модель и проверьте ее работу.

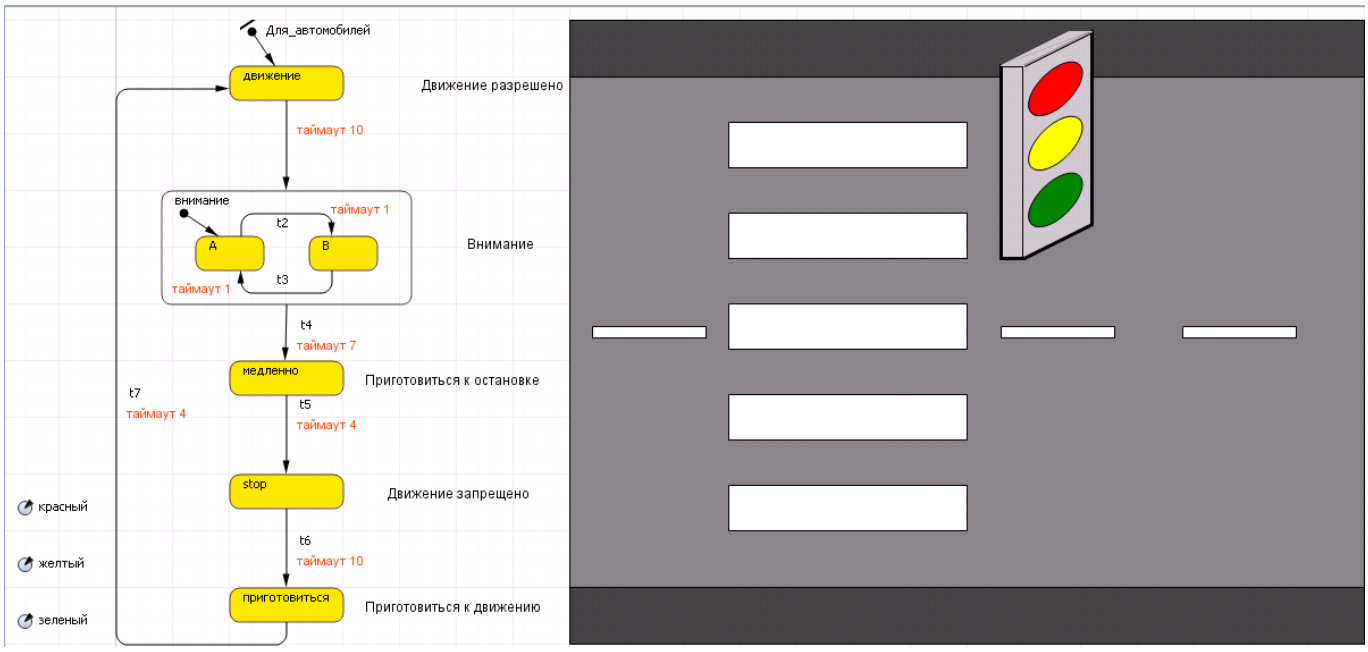


Рис. 3.7

2.4. СРАБАТЫВАНИЕ ПЕРЕХОДА ПО СИГНАЛУ

Добавим к нашей модели второй светофор, для пешеходов. О н будет иметь два сигнала, зеленый и красный, и три состояния: идите (зеленый), внимание (мигающий зеленый) и стойте (красный). Добавим в модель два булевских параметра `стойте` и `идите`, их значениями будет управлять второй стейтчарт – для пешеходов. Создадим этот стейтчарт на той же диаграмме класса `Model`, назвав его `Для_пешеходов`, (рис. 3.8).

Поскольку управление светофором пешеходов похоже на управление светофором автомобилей, новый стейтчарт можно построить копированием и изменением уже построенного стейтчарта `Для_автомобилей`. Выделите 3 состояния стейтчарта `Для_автомобилей` и вставьте их в другое место диаграммы. Эти элементы скопируются, и им автоматически будут даны уникальные имена, чтобы не было конфликта имен. Переименуйте состояния стейтчарта `Для_пешеходов`, дорисуйте недостающий переход `t12` и перенесите начало диаграммы на состояние `Стойте`, как показано на рис. 5.8.

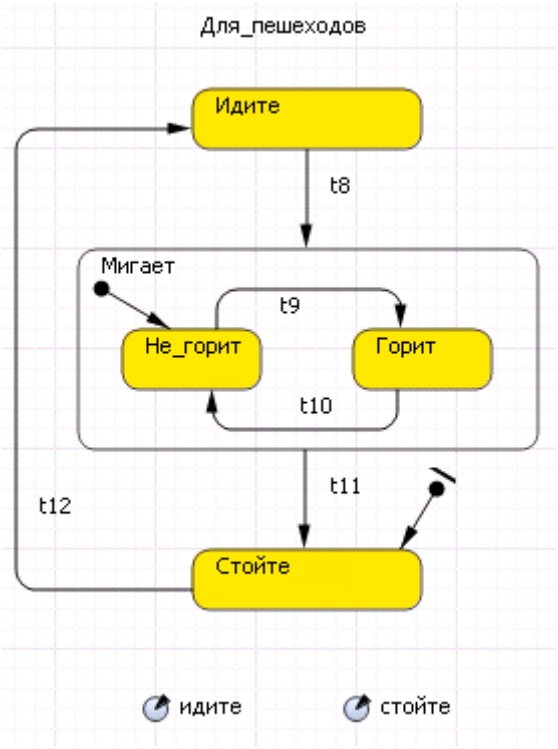


Рис. 3.8

Измените параметры в полях **Дейс твие при входе** и **Действие при выходе** в свойствах состояний стейтчарта `Для_пешеходов`. Теперь наш стейтчарт должен управлять параметрами `идите` и `стойте`, которые, в свою очередь, будут управлять зажиганием света именно пешеходного светофора.

Настроим условия срабатывания переходов стейтчартов между состояниями. Для обеспечения безопасной работы пешеходного перехода необходимо синхронизировать срабатывания стейтчартов так, чтобы всегда, когда светофор пешеходов находится в состояниях `Идите` или `Мигает`, светофор автомобилей обязательно находился бы в состоянии `stop`. Для этого можно подобрать подходящие таймауты срабатывания переходов стейтчарта, но при каждом изменении модели, придется эти таймауты подбирать снова и снова. Более разумно синхронизировать стейтчарты, посылая специальные разрешающие сигналы из одного стейтчарта в другой.

В нашей модели стейтчарты будут обмениваться следующими сигн алами: `АВТОМОБИЛИ` и `ПЕШЕХОДЫ`. В стейтчарте `Для_пешеходов` переход `t12` будет срабатывать когда получен сигнал `ПЕШЕХОДЫ`, который будет генерироваться в стейтчарте `Для_автомобилей` при переходе `t5` в состояние `stop`. В свою очередь, в стейтчарте `Для_автомобилей` переход `t6` будет срабатывать когда получен сигнал `АВТОМОБИЛИ`, который генерируется в стейтчарте `Для_пешеходов` при переходе `t11` в состояние `Стойте`, рис. 3.9.

В AnyLogic есть несколько способов передачи сообщения в диаграмму состоя ний. В нашей модели мы будем использов ать метод `fireEvent( )`, к оторый должен вызываться в том стейтчарте, которому предназначено сообщение. То есть, если из некоего объекта мы хотим послать сообщение стейтчарту, то нужно в этом объекте написать команду:

```
стейтчарт.fireEvent(сообщение)
```

Поэтому, в поле **Действие** перехода `t5` стейтчарта `Для_автомобилей` нужно вставить команду:

```
Для_пешеходов.fireEvent("ПЕШЕХОДЫ"),
```

в такое же поле перехода `t11` стейтчарта `Для_пешеходов` вставьте команду:

```
Для_автомобилей.fireEvent("АВТОМОБИЛИ")
```

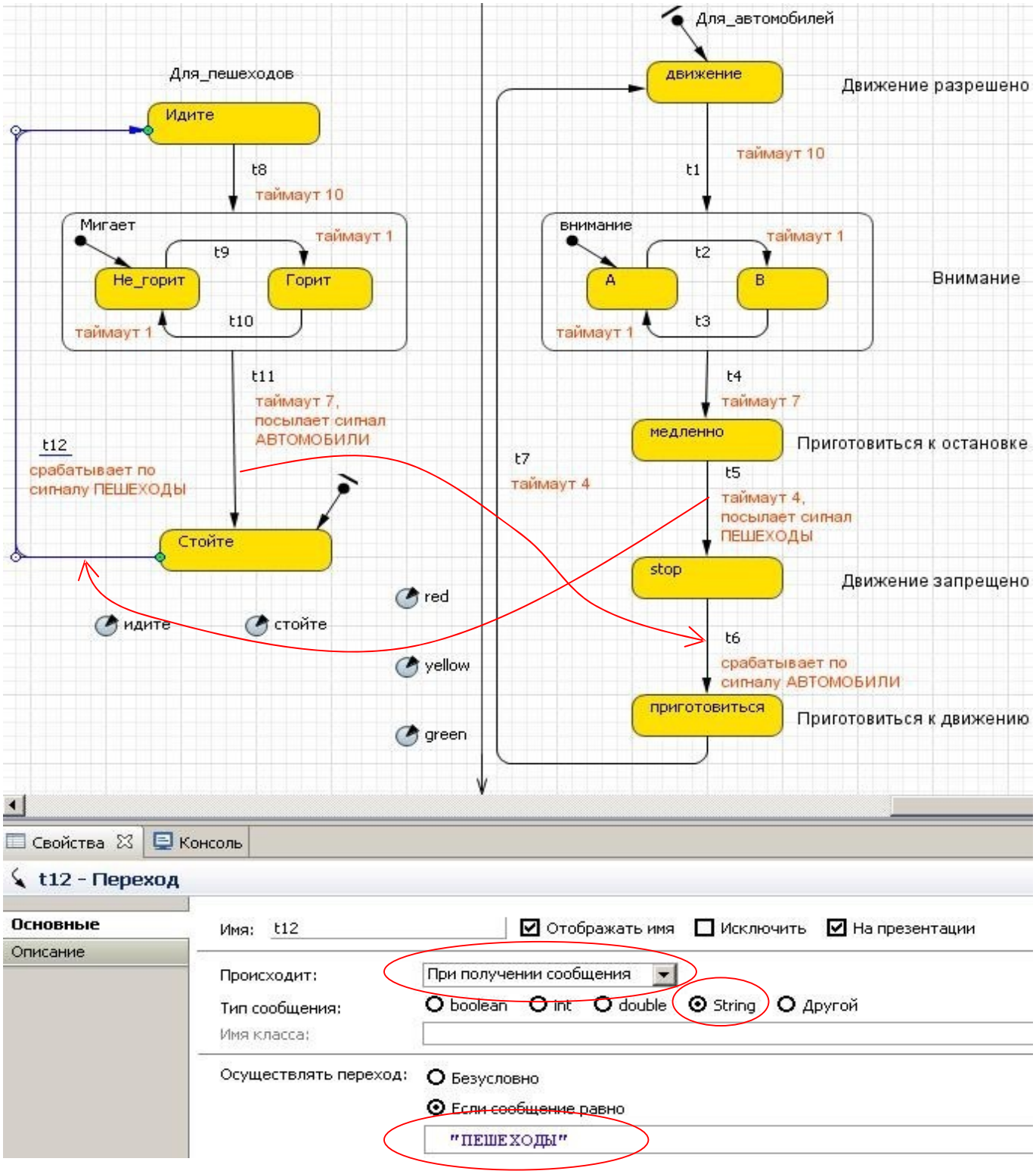


Рис. 3.9

Таким образом, каждый из светофоров будет информировать другого о своем переходе в состояние запрещения движения, как пешеходов, так и автомобилей.

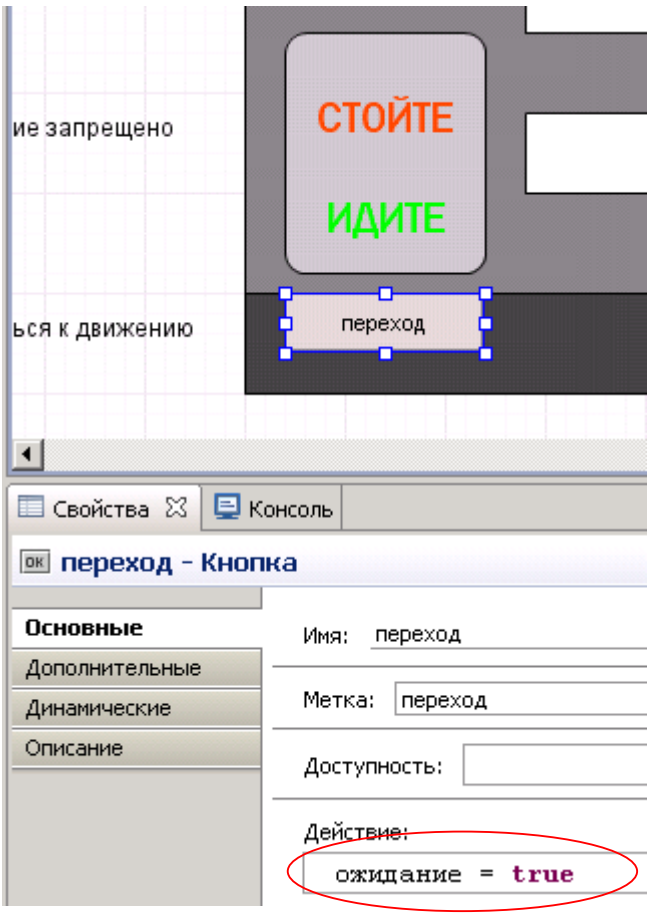
Для срабатывания перехода стейтчарта при получении нужного сообщения, в стейтчарте `Для_пешеходов` в поле **Происходит** окна свойств перехода `t12` выберите вариант **При получении сообщения**, укажите тип сообщения `String`, а в поле **Осуществлять переход** выберите `Если сообщение равно` и введите "`ПЕШЕХОДЫ`", рис. 3.9.

Аналогично, для срабатывания перехода автомобильного стейтчарта по сигналу от пешеходного стейтчарта в стейтчарте `Для_автомобилей` в поле **Происходит** окна свойств перехода `t6` выберите вариант **При получении сообщения**, укажите тип сообщения `String`, а в поле **Осуществлять переход** выберите `Если сообщение равно` и введите "`АВТОМОБИЛИ`".

Остальные переходы этих стейтчартов будут срабатывать по таймаутам, как и прежде. Проверьте по рисунку 3.9 установленные параметры переходов стейтчартов. Запустите модель на выполнение.

На презентации модели, в дополнение к светофору для автомобилей , следует нарисовать светофор для пешеходов с двумя сигналами – красной надписью СТОЙТЕ и зеленой ИДИТЕ. Динамикой цвета этих надписей будут управлять логические параметры `стоите` и `идите`, которые нужно создать на диаграмме по аналогии с параметрами `красный`, `желтый` и `зеленый`. Продемонстрируйте построенную модель преподавателю.

3.5. СРАБАТЫВАНИЕ ПЕРЕХОДА ПО УСЛОВИЮ



Если пешеходы переходят улицу достаточно редко, то автоматическое переключение светофора будет неоправданно часто прерывать движение автомобилей. В этом случае разумно установить специальную кнопку, при нажатии которой пешеходом, светофор автомобилей остановит на некоторое время движение автомашин. Для этого перетащите мышью элемент **Кнопка** с палитры **Элементы управления** на диаграмму класса активного объекта `Model` рядом с пешеходным светофором, рис. 3.10 Назовите кнопку и ее метку: **переход**.

Определите действие при нажатии на кнопку так, чтобы устанавливался в истину булевый параметр `ожидание`. Для этого нужно записать в поле **Действие** окна свойств этой кнопки команду: `ожидание = true`.

Рис. 3.10

Логический параметр ожидание, конечно же, нужно ввести на диаграмме активного объекта Model с начальным значением false. Этот параметр будет определять, собирается ли пешеход перейти дорогу. Значение этого параметра будем переводить в false каждый раз, как только пешеходный светофор перейдет в состояние Мигает. Для этого нужно записать в поле **Действие при входе** окна свойств состояния Мигает стейтчарта Для_пешеходов команду: ожидание=false.

Таким образом, при нажатии кнопки **переход**, стейтчарт автомобилей «узнает» об ожидающих на переходе пешеходах и переключится из состояния Движение в состояние Внимание и затем в состояние запрещения движения автомобилей.

В случае, если некий злоумышленник будет постоянно нажимать кнопку **переход**, это может полностью парализовать движение автомобилей. Для того чтобы этого не произошло, в стейтчарте Для_автомобилей сделайте иерархическим состояние Движение, с двумя простыми состояниями непрерывное и обычное, как показано на рис. 3.11.

В состоянии непрерывное автомобили будут двигаться до истечения таймаута 10 секунд, невзирая на состояние параметра ожидание, т.е. автомобилям будет гарантированно предоставлено некоторое время для движения, даже если кнопка **переход** будет всегда нажата. После истечения таймаута 10 секунд светофор перейдет в обычное состояние движения, которое может прерываться. Переход T из состояния обычное сработает, когда будет нажата кнопка **переход**, т.е. параметр ожидание будет истинен.

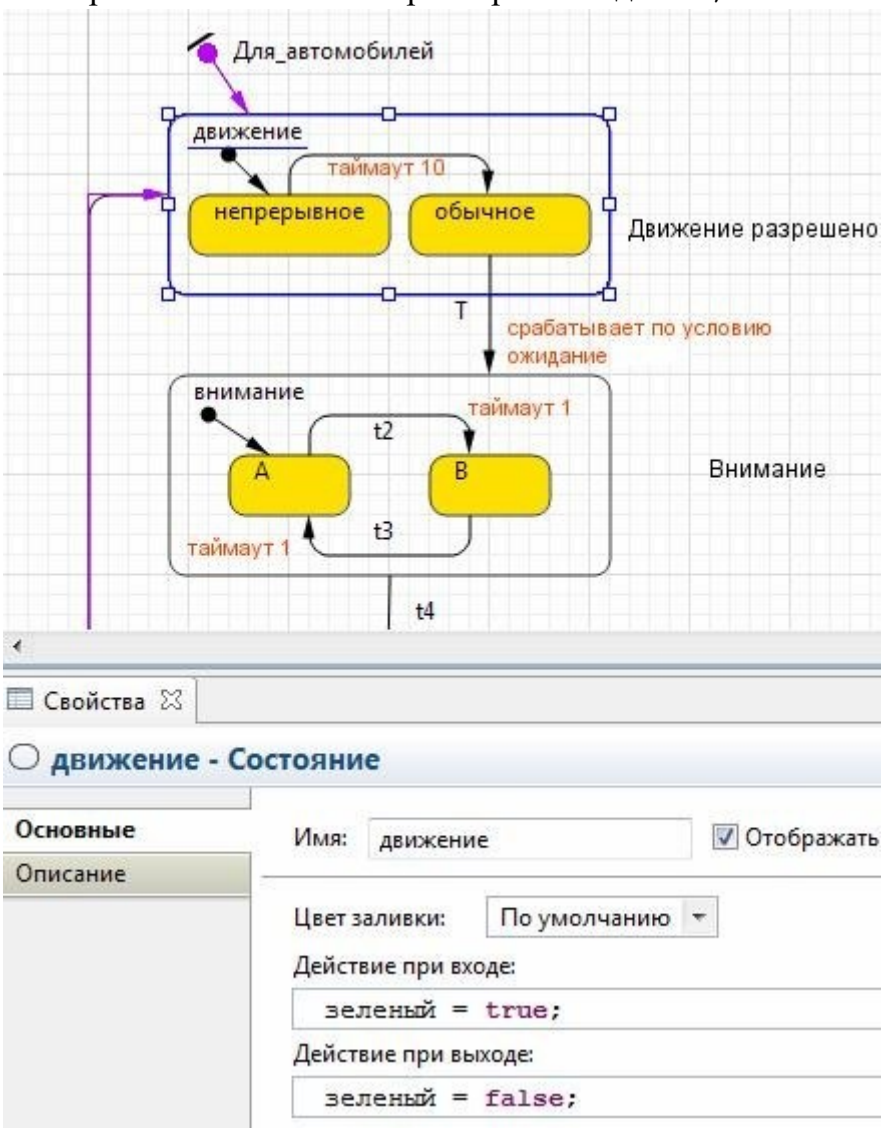


Рис. 3.11

Если кнопка **переход** не нажата (т. е. параметр ожидание имеет значение false), автомобили будут продолжать движение до нажатия этой кнопки. Если нажать кнопку один или более раз, то параметр ожидание станет истинным и автомобильный светофор из состояния обычное перейдет в состояние внимание и затем остановит движение автомобилей.

Проверьте работу модели и продемонстрируйте ее преподавателю.

ЗАДАНИЕ 3

ФИЗИЧЕСКИЙ МАЯТНИК

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

В этом задании мы построим модель маятника. Колебательные движения механических систем широко распространены в технике: качания маятников, движения поршней двигателей внутреннего сгорания, колебания струн, стержней и пластин, вибрации двигателей, фундаментов и множество других подобных процессов.

3.1. СОДЕРЖАТЕЛЬНАЯ ПОСТАНОВКА ЗАДАЧИ

Тело единичной массы прикреплено к неподвижному кронштейну с помощью нерастяжимой и несжимаемой нити длиной `l`. Исследовать колебательные движения тела.

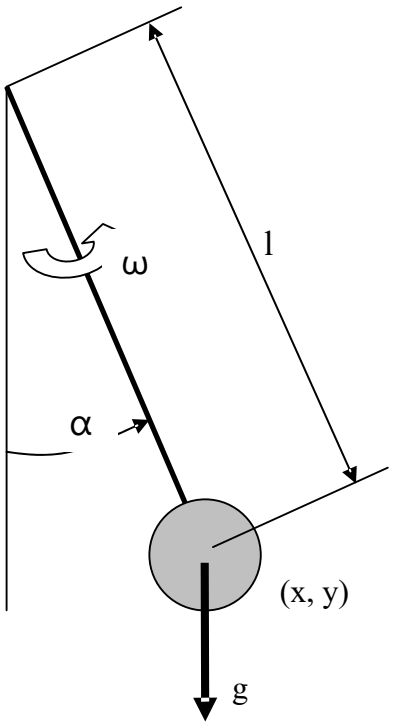


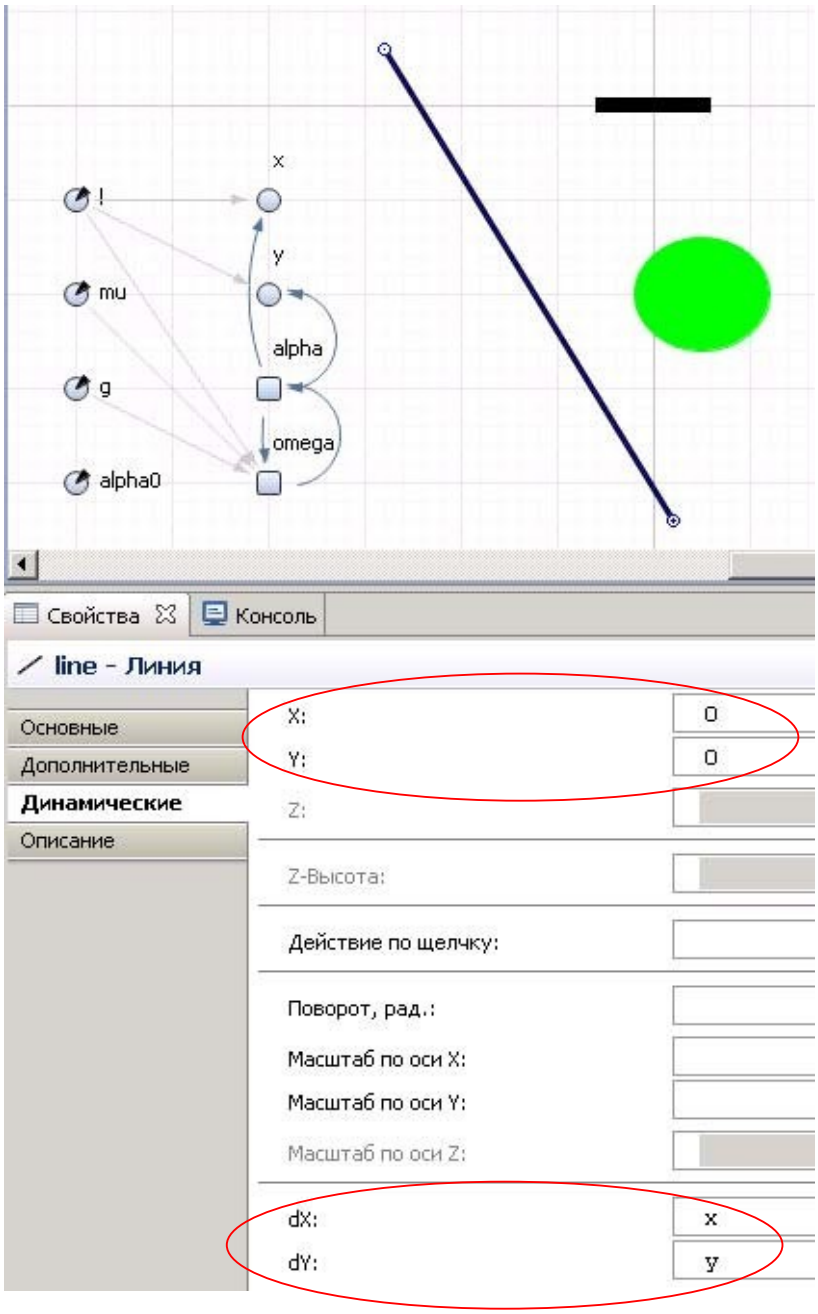
Рис. 1.1

3.2. КОНЦЕПТУАЛЬНАЯ ПОСТАНОВКА ЗАДАЧИ

Примем следующие предположения:

- объектом исследования является тело массой 1, принимаемое за материальную точку;
- движение тела подчиняется второму закону Ньютона;
- тело находится под действием трех сил: силы тяжести `mg`, реакции натяжения нити и силы сопротивления воздуха, пропорциональной квадрату скорости движения;
- тело совершает колебательные движения, так как сила тяжести `mg` уравновешивается вертикальной составляющей реакции натяжения нити;

3.3. МАТЕМАТИЧЕСКАЯ ПОСТАНОВКА ЗАДАЧИ



С математической точки зрения имеем задачу Коши.

$$\frac{d\alpha}{dt} = \omega; \qquad \frac{d\omega}{dt} = -\frac{g \cdot \sin(\alpha)}{l} - \mu \cdot \omega^2;$$

$$x = l \cdot \sin(\alpha); \qquad y = l \cdot \cos(\alpha)$$

$$\alpha(0) = \alpha_0; \qquad \omega(0) = 0$$

Здесь α - текущий угол отклонения маятника от вертикали, ω - его угловая скорость, μ - коэффициент сопротивления среды (будем считать, что сопротивление среды пропорционально квадрату угловой скорости).

3.4. ОПИСАНИЕ МОДЕЛИ

Необходимо построить модель по вышеприведенным формулам. Модель должна содержать две переменных состояния `alpha` и `omega` и три параметра `l`, `mu` и `g`, а также начальное значение переменной `alpha`, которая задается параметром с именем `alpha0`. Переменные `x` и `y` задают координаты центра масс маятника, их значения можно вычислить по формулам, в соответствии со схемой рис. 1.1.

Рис. 1.2

В корневом объекте модели создайте четыре переменных: `x`, `y`, `alpha` и `omega`. Переменные `x` и `y` следует задать формулами, а `alpha` и `omega` – интегралами, в

соответствии с формулами п. 1.3. Начальное значение угловой скорости `omega` равно 0. Четыре параметра `l, mu, g` и `alpha0` так же необходимо создать на диаграмме класса активного объекта.

Обратите внимание на уравнение $\frac{d\omega}{dt} = -\frac{g \cdot \sin(\alpha)}{l} - \mu \cdot \omega^2$, при возведении ω в квадрат член уравнения $-\mu * \omega^2$ всегда будет отрицательным независимо от направления движения маятника (знака переменной ω), т.е. будет потеряно направление действия силы сопротивления, что приведет к ошибке. Необходимо видоизменить это уравнение так, чтобы знак угловой скорости не потерялся: $\frac{d\omega}{dt} = -\frac{g \cdot \sin(\alpha)}{l} - \mu \cdot \omega \cdot |\omega|$.

3.5. **ПРЕЗЕНТАЦИЯ**

В презентации определите четыре области. В центральной с помощью двух линий и овала нарисуйте маятник. У линии, изображающей нить, один конец всегда имеет координаты (0,0), этот конец нити привязан к неподвижной опоре. Второй конец нити имеет координаты `x` и `y`, этот конец нити привязан к маятнику.

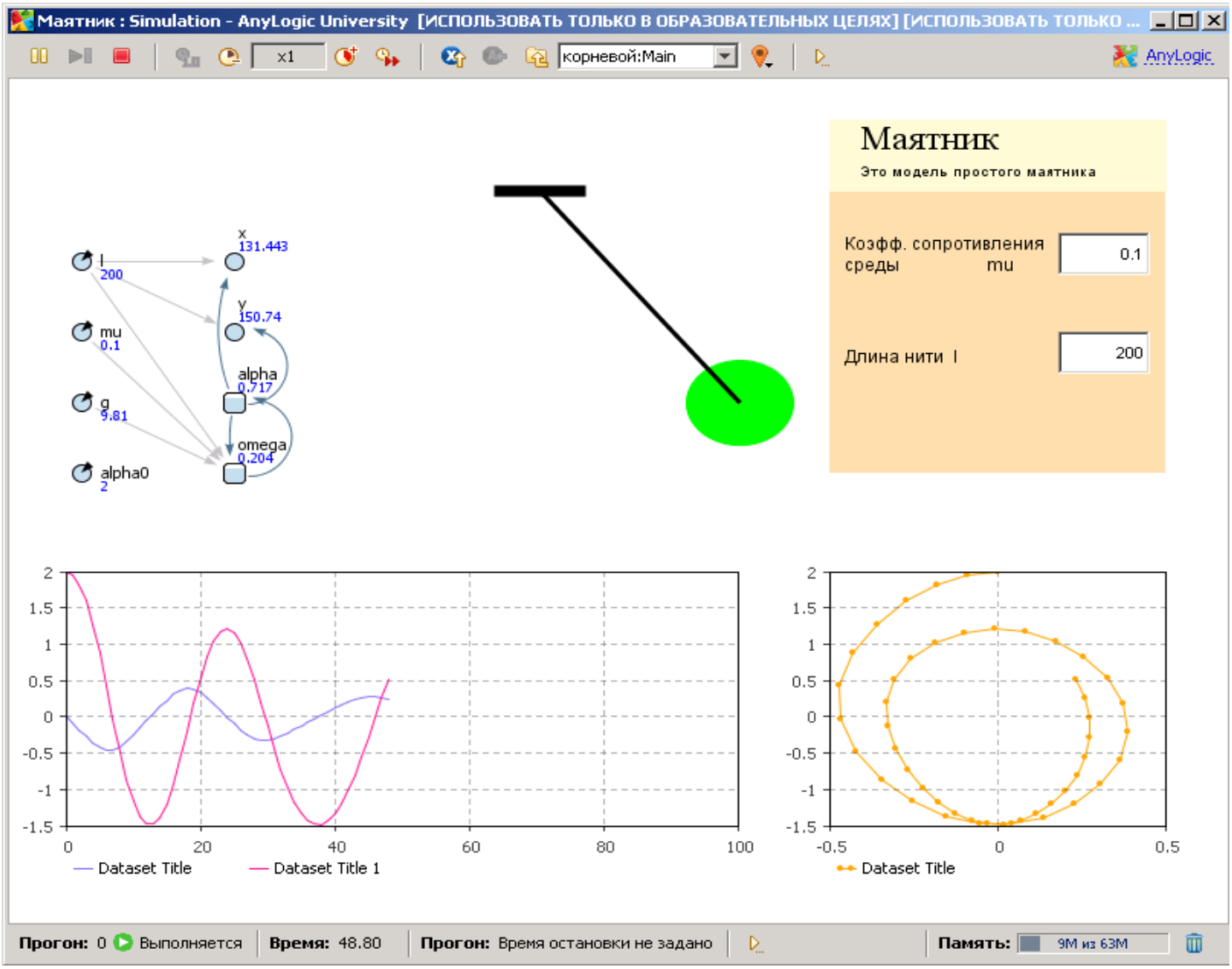


Рис. 1.3

Соответственно, на закладке **Динамические** панели свойств линии, отображающей нить, в поле значений координат начала нити X и Y введите 0, в поле значений координат конца

нити dX и dY введите величины `x` и `y`, рис.1.2. Это приведет к тому, что при работе модели концы отрезка всегда будут находиться в точках с координатами (0, 0) и (x, y). У овала во вкладке с динамическими значениями координат установите `x` и `y`.

Третья область должна содержать: название модели и небольшой поясняющий текст.

Нижняя часть третьей области должна включать два специальных поля для ввода данных: **Текстовое поле**, более известное как поле редактора или editBox, В эти поля при работе модели можно вводить новые значения параметров `l` и `mu`, изменяя их в процессе выполнения модели.

В нижней части диаграммы класса активного объекта разместите графики зависимости `alpha` и `omega` от времени, и фазовую диаграмму зависимости `alpha` от `omega`.

Постройте модель как показано на рис. 1.3 и продемонстрируйте ее преподавателю.